

Министерство науки и высшего образования Российской Федерации
Московский физико-технический институт (государственный университет)

Физтех-школа радиотехники и компьютерных технологий
Кафедра микропроцессорных технологий в интеллектуальных системах управления

Выпускная квалификационная работа бакалавра

Реализация векторизации ациклических участков кода по методу Superword-Level Parallelism в компиляторе языка программирования Go

Автор:

Студент Б01-2076 группы
Самойлов Арсений Сергеевич

Научный руководитель:

Петушков Игорь Вячеславович

Научный консультант:

Маркин Алексей Леонидович



Москва 2026

Аннотация

Реализация векторизации ациклических участков кода по методу Superword-Level Parallelism в компиляторе языка программирования Go
Самойлов Арсений Сергеевич

В работе рассматривается задача автоматической векторизации программного кода в компиляторе языка Go. Отсутствие в стандартном компиляторе оптимизаций, использующих SIMD-инструкции, ограничивает производительность приложений. В работе предложен и реализован проход SLP-векторизации, адаптированный к SSA представлению и архитектурным ограничениям Go: явному моделированию памяти, отсутствию сложных анализов и требованию быстрой компиляции. Разработанный векторизатор выполняет поиск изоморфных независимых операций, расширение групп, валидацию и генерацию 128-битных векторных инструкций для архитектур x86-64 (SSE/AVX) ARM64 (NEON). Экспериментальная оценка на синтетических тестах и реальных приложениях показала ускорение до XX% при незначительном увеличении времени компиляции (порядка 1%). Работа демонстрирует принципиальную возможность и практическую пользу внедрения SLP-векторизации в основной компилятор Go.

Содержание

1	Введение	5
1.1	Мотивация	5
1.2	Постановка задачи	5
2	Автоматическая векторизация программного кода	6
2.1	SIMD как аппаратная основа векторизации	6
2.2	Подходы к автоматической векторизации	7
2.3	Цикловая векторизация	7
2.4	SLP-векторизация	7
2.5	Сравнение подходов	8
2.6	Метод Superword-Level Parallelism	8
2.6.1	Основные этапы алгоритма	8
2.7	Особенности метода SLP	9
2.7.1	Причины выбора SLP для компилятора Go	9
2.8	Выводы по главе	10
3	Обзор существующих решений	11
3.1	Реализация в LLVM	11
3.1.1	Архитектура раннего SLP-векторизатора LLVM	12
3.1.2	Построение и оценка дерева выражений	12
3.1.3	Генерация векторного кода	12
3.1.4	Основные ограничения ранней реализации	12
3.1.5	Значение для выполненной работы	13
3.2	Ограничения переноса существующих решений в компилятор Go	13
3.2.1	Ограниченный набор аналитических проходов	13
3.2.2	Явное моделирование памяти в Go SSA	13
3.2.3	Философия быстрой компиляции	14
3.2.4	Итог	14
4	Компилятор Go	15
4.1	Существующие реализации компилятора Go и проблема векторизации	15
4.2	SSA-представление	15
4.3	Оптимизационный конвейер	16
4.4	Особенности SSA, значимые для SLP-векторизации	17
4.5	Архитектурные ограничения для внедрения SLP	17
4.5.1	Явное моделирование памяти	17
4.5.2	Ограниченная аналитическая инфраструктура	18
4.5.3	Требования к времени компиляции	18
4.5.4	Особенности рантайма языка	18
4.6	Выбор точки интеграции	19
4.7	Выводы по главе	19
5	Проектирование SLP-векторизатора	21
5.1	Требования и ограничения для Go-векторизатора	21
5.2	Общая архитектура	21
5.3	Поиск кандидатов для векторизации	24
5.4	Построение графа зависимостей и расширение групп	24
5.5	Валидация и планирование	25
5.5.1	Проверка внешних использований	25

5.5.2	Проверка замкнутости операндов	25
5.5.3	Проверка когерентности линий	25
5.5.4	Проверка согласованности исходных операндов	25
5.6	Легализация и генерация кода	25
5.7	Принятые компромиссы и направления развития	26
6	Реализация	28
6.1	Инструментарий и среда разработки	28
6.2	Базовые структуры данных и проверки	28
6.3	Проход <code>slp</code> : главный управляющий цикл	29
6.4	Этапы анализа	29
6.4.1	Поиск начальных пар	29
6.4.2	Расширение групп	29
6.4.3	Комбинирование	30
6.4.4	Разбиение на компоненты связности	30
6.5	Валидация и планирование	30
6.5.1	Валидация	30
6.5.2	Планирование	31
6.6	Генерация векторного SSA-кода	31
6.6.1	Легализация	31
6.6.2	Создание векторных инструкций	31
6.6.3	Замена скалярных использований	31
6.7	Тестирование корректности	32
7	Экспериментальная оценка реализации	33
7.1	Цели и методика оценки	33
7.2	Выбор тестовых наборов	33
7.2.1	Набор синтетических тестов	33
7.2.2	Трассировщик лучей	34
7.2.3	<code>etcd</code>	34
7.2.4	<code>Sweet</code>	34
7.3	Результаты измерения производительности	34
7.3.1	Набор синтетических тестов	34
7.3.2	Трассировщик лучей	35
7.3.3	<code>etcd</code>	35
7.4	<code>Sweet</code>	35
7.4.1	Влияние на время компиляции	35
7.5	Сравнение с <code>gollvm</code>	35
7.6	Обсуждение ограничений и не векторизованных случаев	35
7.7	Направления будущей работы	36
8	Заключение	37

1 Введение

1.1 Мотивация

Одним из широко используемых аппаратных механизмов повышения производительности программ является использование SIMD-параллелизма (Single Instruction Multiple Data), при котором одна процессорная инструкция выполняет операцию над несколькими элементами данных одновременно.

Для эффективного использования SIMD-инструкций в компиляторах применяются методы автоматической векторизации, преобразующие скалярные операции в их векторные аналоги. Однако автоматическая векторизация в главной реализации компилятора Go отсутствует. Это приводит к недоиспользованию аппаратных возможностей процессоров, снижая потенциальную производительность программ.

Одним из подходов к автоматической векторизации является метод Superword-Level Parallelism (SLP). В отличие от цикловой векторизации, ориентированной на анализ и преобразование циклических конструкций, SLP-векторизация направлена на поиск независимых однотипных операций внутри ациклических участков программы. Такой подход требует менее сложного анализа зависимостей, проще интегрируется в компилятор и обладает меньшими накладными расходами на этапе компиляции.

Выбор метода SLP особенно актуален для компилятора Go, поскольку реализация сложных цикловых оптимизаций требует существенного расширения существующей инфраструктуры анализа. При этом SLP-векторизация позволяет получить практическое ускорение на ряде классов задач при относительно невысокой сложности реализации.

1.2 Постановка задачи

Объектом исследования является подсистема оптимизаций на уровне SSA (Static Single Assignment) представления компилятора языка Go.

Предметом исследования являются методы автоматической векторизации ациклических участков кода на основе подхода Superword-Level Parallelism (SLP).

Целью работы является разработка и реализация прототипа механизма автоматической SLP-векторизации ациклических участков кода в компиляторе Go.

Для достижения поставленной цели в работе решаются следующие **задачи**:

- исследовать существующие методы автоматической векторизации;
- проанализировать архитектуру оптимизационного конвейера компилятора Go;
- разработать алгоритм выявления векторизуемых групп операций;
- реализовать преобразования SSA-представления для генерации векторных операций;
- провести экспериментальную оценку корректности и эффективности разработанного решения.

Практическая значимость работы заключается в разработке прототипа механизма автоматической векторизации для компилятора Go, демонстрирующего возможность интеграции SLP-преобразований в существующий компилятор. Полученные результаты могут быть использованы как основа для дальнейшего развития оптимизирующих подсистем компилятора.

2 Автоматическая векторизация программного кода

2.1 SIMD как аппаратная основа векторизации

SIMD (Single Instruction Multiple Data) [1] представляет собой модель вычислений, при которой одна инструкция выполняет одинаковую операцию над несколькими элементами данных, размещёнными в векторных регистрах. Данный подход позволяет повысить производительность за счёт параллельной обработки независимых данных.

Векторный регистр состоит из нескольких одинаковых по размеру элементов, называемых линиями. Каждая линия содержит одно значение обрабатываемого типа данных, а стандартная SIMD-инструкция выполняет одну и ту же операцию одновременно над всеми линиями регистра. Число линий определяется шириной регистра и размером элемента данных. Например, 128-битный регистр может содержать четыре 32-битных целых числа или два 64-битных числа.

Простейший пример SIMD-вычислений — поэлементное сложение двух массивов:

$$c_i = a_i + b_i, \quad i = 0, \dots, n-1$$

При использовании SIMD вычисления выполняются над вектором из m элементов одновременно:

$$\mathbf{c}_i = \mathbf{a}_i + \mathbf{b}_i, \quad \mathbf{a}_i, \mathbf{b}_i, \mathbf{c}_i \in \mathbb{R}^m$$

где m определяется шириной векторного регистра и типом данных.

Современные процессорные архитектуры поддерживают SIMD-вычисления через специализированные наборы инструкций:

- ARM: NEON и SVE;
- x86: SSE, AVX, AVX2, AVX-512.

По функциональному назначению SIMD-инструкции можно условно разделить на следующие классы:

- арифметические операции (сложение, умножение и т.д.);
- логические операции;
- операции перестановки и упаковки данных (shuffle, permute);
- операции загрузки и сохранения векторных данных;
- редукционные операции.

Эффективное применение SIMD требует выполнения ряда условий:

- отсутствие зависимостей по данным между обрабатываемыми элементами;
- согласованность типов и размеров операндов;
- возможность группировки скалярных операций в векторные пакеты.

Несмотря на наличие аппаратной поддержки, использование SIMD-инструкций на уровне прикладного программирования связано с рядом трудностей, включая зависимость от конкретной архитектуры и необходимость явного управления векторными регистрами.

Поэтому важную роль играют методы автоматической векторизации, позволяющие компилятору выявлять участки кода, пригодные для преобразования в SIMD-форму.

2.2 Подходы к автоматической векторизации

Автоматическая векторизация представляет собой класс оптимизаций компилятора, направленных на преобразование скалярного программного кода в эквивалентную форму, использующую SIMD-инструкции.

В современных компиляторах можно выделить два основных подхода к автоматической векторизации:

- цикловая векторизация;
- SLP-векторизация.

2.3 Цикловая векторизация

Цикловая векторизация ориентирована на выявление параллелизма между итерациями циклов. Рассмотрим пример:

```
for i := 0; i < n; i++ {  
    c[i] = a[i] + b[i]  
}
```

Данный цикл не содержит межитерационных зависимостей, что позволяет выполнять операции над несколькими итерациями одновременно с использованием SIMD-инструкций.

Векторизация преобразует последовательность скалярных операций в последовательность векторных операций, каждая из которых обрабатывает несколько элементов массива одновременно.

Эффективность цикловой векторизации определяется возможностью доказать отсутствие межитерационных зависимостей. Для этого компилятор выполняет:

- анализ псевдонимов памяти (alias analysis) [2, 3];
- анализ межитерационных зависимостей (loop-carried dependence analysis) [4];

А также вспомогательные преобразования:

- раскрутку цикла (loop unrolling) [5, 6];
- выделение остаточных итераций (loop peeling) [7];

Реализация таких механизмов требует развитой инфраструктуры анализа зависимостей и существенно усложняет оптимизационный конвейер компилятора.

2.4 SLP-векторизация

SLP-векторизация ориентирована на выявление параллелизма внутри базовых блоков программы. В отличие от цикловой векторизации, данный подход не использует межитерационный анализ, а работает с зависимостями внутри ациклических участков кода.

Алгоритм формирует группы изоморфных независимых скалярных операций, которые могут быть преобразованы в одну векторную инструкцию.

Например:

```
c[0] = a[0] + b[0]  
c[1] = a[1] + b[1]
```

может быть преобразовано в одну SIMD-инструкцию сложения векторов.

SLP особенно эффективен в случаях, когда параллелизм не выражен явно через циклы.

2.5 Сравнение подходов

Цикловая векторизация и SLP-векторизация различаются по области применимости и требованиям к анализу зависимостей.

Цикловая векторизация требует анализа межитерационных зависимостей. SLP-векторизация ограничена областью базового блока, но требует менее сложной инфраструктуры анализа.

Векторизация циклов обычно обеспечивает более высокий потенциальный выигрыш, однако её реализация требует развитой системы анализа зависимостей и сложной трансформации циклов. SLP-векторизация, напротив, обладает меньшей сложностью реализации.

Для компилятора Go это особенно важно, поскольку в нём отсутствует развитая инфраструктура цикловых оптимизаций, а требования к времени компиляции являются критичными.

2.6 Метод Superword-Level Parallelism

Метод Superword-Level Parallelism был предложен в работе Samuel Larsen и Saman Amarasinghe [8] как способ выявления параллелизма на уровне независимых скалярных операций внутри базовых блоков программы. Основная идея метода заключается в объединении нескольких однотипных независимых скалярных операций в одну векторную.

2.6.1 Основные этапы алгоритма

Классический алгоритм SLP-векторизации [8] состоит из нескольких последовательных этапов, выполняемых в пределах одного базового блока.

1. **Поиск начальных пар.** Алгоритм сканирует инструкции базового блока и ищет пары соседних обращений к памяти – смежные загрузки или сохранения данных. Такие операции могут послужить естественным началом и/или завершением параллельных последовательностей вычислений.
2. **Расширение групп.** От найденных начальных пар алгоритм итеративно строит новые группы, анализируя поток данных:
 - *Расширение вверх:* для каждой пары инструкций в группе анализируются их аргументы – если находятся две однотипные независимые операции, они могут образовать новую пару.
 - *Расширение вниз:* рассматриваются операции, использующие результат исходных операций. Среди нескольких возможных вариантов выбирается та пара использований, которая приносит наибольшую выгоду с точки зрения модели стоимости.

Процесс продолжается, пока находятся новые пары, удовлетворяющие условиям независимости и изоморфности операций.

3. **Объединение пар в группы.** После завершения расширения ищутся пары, которые можно объединить. Если в двух парах есть общая операция, то их можно объединить в тройки, четвёрки и т.д., формируя потенциальные векторные регистры большей ширины.

4. **Планирование.** Векторные инструкции должны располагаться в корректном порядке с учётом зависимостей по данным. Выполняется топологическая сортировка групп: ни одна группа не может быть помещена в выходной поток раньше тех групп, от которых она зависит через аргументы.
5. **Генерация векторного кода.** Для каждой группы формируются соответствующие векторные инструкции целевой архитектуры. При необходимости группы разбиваются до поддерживаемой аппаратурой ширины. Если какая-то часть промежуточного результата используется вне векторизованного фрагмента, то нужное значение скаляризуется – вставляются специальные операции извлечения отдельных данных из векторного регистра в обычный.

Такой набор этапов гарантирует корректность векторизации и позволяет последовательно расширять функциональность, начиная с простых парных операций.

2.7 Особенности метода SLP

Метод SLP обладает следующими особенностями по сравнению с цикловой векторизацией:

- ограниченная область анализа (в пределах базового блока);
- отсутствие необходимости анализа межитерационных зависимостей;
- относительно низкая сложность реализации в компиляторе;
- хорошая интеграция в SSA-представление.

Данные свойства делают SLP подходом, хорошо подходящим для компиляторов с ограниченной инфраструктурой оптимизаций.

Однако, метод эффективен только при наличии явно выраженных групп независимых однотипных операций. Если вычисления имеют сложную структуру зависимостей или требуют существенного переупорядочивания данных, эффективность SLP снижается.

2.7.1 Причины выбора SLP для компилятора Go

Выбор метода SLP для реализации в компиляторе Go обусловлен рядом факторов:

- Данный подход хорошо согласуется с существующей архитектурой компилятора.
- Реализация SLP требует существенно меньшего объёма инфраструктурных изменений по сравнению с цикловой векторизацией.
- Алгоритм обладает умеренной вычислительной сложностью, что соответствует требованию сохранения малого времени компиляции.

Таким образом, SLP представляет собой практически обоснованный компромисс между сложностью реализации и потенциальным приростом производительности.

2.8 Выводы по главе

В данной главе были рассмотрены аппаратные основы SIMD-вычислений, основные подходы к автоматической векторизации и детали метода Superword-Level Parallelism. Показано, что SLP-векторизация, в отличие от цикловой, не требует анализа итерационных зависимостей и сложных трансформаций циклов. Это делает её практически применимым подходом для интеграции в компилятор Go: умеренная алгоритмическая сложность сочетается с возможностью ускорять ациклический код, типичный для многих серверных и системных программ на Go.

3 Обзор существующих решений

Метод SLP получил широкое распространение в современных оптимизирующих компиляторах. Наиболее известные промышленные реализации присутствуют в инфраструктурах LLVM и GCC. За время развития данных проектов алгоритмы SLP-векторизации существенно усложнились и были дополнены многочисленными эвристиками, ориентированными на повышение качества генерируемого кода и поддержку широкого спектра архитектур.

3.1 Реализация в LLVM

SLP-векторизатор был добавлен в LLVM в 2013 году. Первоначальная реализация представляла собой сравнительно компактный алгоритм, ориентированный на поиск групп изоморфных инструкций и построение деревьев векторизации внутри базовых блоков.

Основными возможностями ранней реализации являлись:

- объединение однотипных арифметических операций;
- базовая модель оценки прибыльности преобразований.
- векторизация простейших арифметических операций;

Дальнейшее развитие LLVM привело к существенному расширению функциональности SLP-векторизатора. Современная реализация поддерживает:

- частичную векторизацию;
- сложные стратегии переупорядочивания операций и элементов;
- векторизацию редукций;
- векторизацию между базовыми блоками;

В результате кодовая база современного SLP-векторизатора LLVM значительно выросла и включает большое количество эвристик, ориентированных на конкретные архитектурные особенности.

В рамках данной работы для анализа была рассмотрена ранняя реализация SLP-векторизации, представленная в LLVM версии 3.3. Выбор именно данной версии обусловлен несколькими причинами:

1. Ранняя реализация содержит базовые алгоритмические идеи метода в относительно компактной и понятной форме.
2. Она лишена значительного количества инженерных усложнений, накопленных в современных версиях LLVM.
3. Архитектурные решения раннего SLP-векторизатора лучше подходят для адаптации в условиях ограниченной инфраструктуры анализа, характерной для компилятора Go.

Таким образом, использование реализации LLVM 3.3 для анализа позволяет опираться на проверенный промышленный подход, сохраняя при этом приемлемую сложность.

3.1.1 Архитектура раннего SLP-векторизатора LLVM

В основе алгоритма лежит обход базового блока, начинающийся с поиска цепочек последовательных инструкций сохранения значения в память. Для этого используется анализатор скалярной эволюции (ScalarEvolution [9]), определяющий, обращаются ли две инструкции к смежным областям памяти. Если две записи сохраняют однотипные значения по адресам, отличающимся ровно на размер элемента, они считаются последовательными.

Найденные пары объединяются в цепочки: пока следующее сохранение является смежным с предыдущим и ещё не помечено как векторизованное, оно добавляется в группу. Далее для каждой цепочки запускается попытка векторизации скользящим окном, размер которого определяется целевой шириной векторного регистра.

3.1.2 Построение и оценка дерева выражений

Для выбранного окна сохранений строится дерево выражений путём рекурсивного подъёма по аргументам. Алгоритм поднимается к операциям, породившим сохраняемые значения. Рекурсивно формируется набор инструкций-предшественников, проверяется их изоморфизм и допустимость упаковки.

Процесс управляется классом `BoUpSLP`, который на этапе анализа стоимости просматривает дерево с ограничением глубины. Все инструкции нумеруются для контроля порядка, а также фиксируется, в какой «линии» (позиции внутри будущего вектора) используется каждое значение. Если одна и та же инструкция оказывается востребованной в разных линиях, она помечается как подлежащая обязательной скаляризации — её результат не может быть частью векторного регистра без дублирования, что обычно дорого.

Стоимость векторизации рассчитывается рекурсивно: для каждого узла дерева стоимость скалярных эквивалентов сравнивается со стоимостью векторной операции, получаемой через интерфейс `TargetTransformInfo`. Учитываются затраты на загрузку/сохранение, арифметические операции и возможные вставки/извлечения элементов. Дополнительно проверяется, не мешают ли промежуточные инструкции, работающие с памятью, безопасному перемещению ранних загрузок к месту размещения векторизованных операций (анализ `isUnsafeToSink`).

3.1.3 Генерация векторного кода

Если модель стоимости показывает выгоду, запускается `vectorizeTree`. Рекурсивно, начиная с листьев дерева, создаются векторные инструкции. Для случаев, когда дерево не завершается сохранением, существует метод `vectorizeArith`, который строит вектор, а затем вставляет операции извлечения элементов для каждого исходного использования, сохраняя корректность программы.

3.1.4 Основные ограничения ранней реализации

Ранний SLP-векторизатор LLVM обладает рядом естественных ограничений, многие из которых были сняты в более поздних версиях:

- **Один базовый блок:** векторизация не переходит через границы блоков; PHI-узлы не поддерживаются.
- **Фиксированная векторная ширина:** используется минимальный размер векторного регистра (128 бит), кратность ширине скалярного типа. Нет поддержки частичной упаковки неполных векторов.

- **Правило однократного использования:** если скалярная инструкция имеет несколько использований, она помечается `MustScalarize`, что часто делает векторизацию невыгодной.
- **Глубина рекурсии:** ограничение в несколько уровней рекурсии не позволяет векторизовать глубокие деревья выражений.

3.1.5 Значение для выполненной работы

Ранняя реализация LLVM примечательна тем, что в сжатой форме воплощает ключевые принципы SLP: поиск начальных пар через обращения в память, рекурсивное расширение, оценку стоимости и генерацию векторных инструкций. Её изучение позволяет выделить алгоритмическое ядро, свободное от инженерных наслоений поздних версий.

Вместе с тем, восходящий от сохранений подход плохо согласуется с устройством SSA-формы компилятора Go, где состояние памяти задано явным потоком значений `Memory`. Прямое следование логике LLVM потребовало бы существенной переработки, что выводится за рамки данного этапа и подробно обсуждается в разделе 3.2 (Ограничения переноса существующих решений в компилятор Go).

3.2 Ограничения переноса существующих решений в компилятор Go

Несмотря на наличие работоспособных и проверенных временем реализаций SLP-векторизации, их прямое или близкое к исходному заимствование в компиляторе Go оказывается невозможным. Причины лежат в фундаментальных различиях архитектуры промежуточного представления, состава аналитической инфраструктуры и целевых установок проекта.

3.2.1 Ограниченный набор аналитических проходов

Компилятор LLVM опирается на развитые анализы: точный анализ псевдонимов указателей (aliasing), скалярную эволюцию, анализ зависимостей по данным, анализ циклов. Ранний SLP-векторизатор LLVM активно использует `ScalarEvolution` для проверки смежности адресов и `AliasAnalysis` для оценки безопасности перемещения инструкций памяти.

Инфраструктура компилятора Go, напротив, минимимальна: полноценный анализ псевдонимов отсутствует, отсутствует модель стоимости инструкций. Даже существующие проходы, анализирующие смежность адресов памяти, работают с простыми локальными шаблонами. Перенос даже ранней версии LLVM потребовал бы либо воссоздания значительной части аналитической подсистемы, либо замены точных проверок консервативными эвристиками.

3.2.2 Явное моделирование памяти в Go SSA

В LLVM IR порядок операций с памятью не задан жёстко; две независимые загрузки из заведомо различных областей могут быть переставлены, а `store`-инструкции не связываются виртуальными рёбрами. В Go SSA каждая операция, затрагивающая память, явно принимает и порождает значение типа `Memory`, образуя упорядоченную цепочку состояний. Эта модель, удобная для потокового анализа и понижения до машинного кода, становится серьёзным препятствием для восходящей стратегии SLP, принятой в LLVM. Две потенциально векторизуемые загрузки могут иметь различные входные значения `Memory` из-за наличия между ними записи по гарантированно не пересе-

кающемся адресу, что с точки зрения консервативного анализа делает их зависимыми и не подлежащими упаковке.

3.2.3 Философия быстрой компиляции

Компилятор Go проектировался с приоритетом скорости сборки, поэтому в нём не приветствуются дорогостоящие анализы, способные замедлить трансляцию крупных проектов. Ранний SLP-векторизатор LLVM, при всей своей относительной простоте, уже включает рекурсивный обход дерева с квадратичными проверками. Прямой перенос подобного прохода без адаптации к ограничениям по времени компиляции противоречил бы проектным принципам Go, требуя аккуратного выбора алгоритмических компромиссов.

3.2.4 Итог

Перечисленные факторы в совокупности делают невозможным простое портирование существующей реализации SLP-векторизатора. Вместо этого требуется самостоятельная разработка, опирающаяся на базовые идеи метода, но адаптированная к модели памяти, инфраструктуре анализа и ограничениям для быстрой компиляции, присущим компилятору Go.

4 Компилятор Go

4.1 Существующие реализации компилятора Go и проблема векторизации

Основным (и официально поддерживаемым) компилятором языка Go является `cmd/compile` [10, 11] — самодостаточный компилятор, ориентированный на высокую скорость сборки. Именно он является основным в экосистеме Go. Однако существует и несколько альтернативных реализаций, среди которых наиболее заметна `gollvm` [?] — компилятор, построенный на инфраструктуре LLVM.

На первый взгляд, `gollvm` мог бы полностью снять необходимость разработки собственного векторизатора для Go: LLVM содержит зрелые и активно развиваемые проходы цикловой и SLP-векторизации, способные генерировать высокоэффективный SIMD-код. Тем не менее использование `gollvm` в качестве основного инструмента для внедрения автоматической векторизации в экосистему Go сопряжено с рядом проблем, делающих его неприемлемым на практике:

- **Несовместимость с рантаймом Go.** Стандартный рантайм Go тесно завязан на конкретные свойства кода: точный сборщик мусора опирается на карты указателей, планировщик горутин ожидает определённой структуры стековых фреймов и корректного сохранения/восстановления регистров в точках синхронизации. Поэтому `gollvm` пользуется собственной версией рантайма, которая сильно уступает основной.
- **Отставание от основного компилятора и нестабильность.** `gollvm` является экспериментальным проектом и не входит в официальный набор инструментов Go. Его развитие отстает от основной имплементации: внедрение новых возможностей языка, изменений в спецификации или внутреннего устройства рантайма требует длительной адаптации.
- **Несоответствие требованию быстрой компиляции.** LLVM предоставляет богатый набор глубоких анализов и агрессивных оптимизаций, но ценой значительного увеличения времени компиляции. Сборка `gollvm`'ом может занимать в разы или даже на порядок больше времени по сравнению со стандартным компилятором. Учитывая, что быстрота компиляции является одной из ключевых ценностей языка, переход на LLVM-бэкенд для массового использования маловероятен — это противоречит ожиданиям разработчиков.

Вывод для настоящей работы. Таким образом, хотя `gollvm` демонстрирует потенциальные выгоды от векторизации Go-кода, он не может служить практическим решением для большинства пользователей языка. Единственным путём внедрения автоматической векторизации в Go является разработка соответствующих оптимизационных проходов непосредственно внутри стандартного компилятора с полным учётом его архитектуры, ограничений и требований рантайма. Именно это и составляет предмет данной работы. Далее в главе рассматривается структура и особенности стандартного компилятора Go, значимые для реализации SLP-векторизатора.

4.2 SSA-представление

SSA-представление компилятора Go (пакет `cmd/compile/internal/ssa`) реализовано в виде набора связанных структур данных, ключевыми из которых являются `Block` (базовый блок) и `Value` (инструкция-значение). Каждая функция после построения SSA состоит из графа базовых блоков, внутри каждого из которых располагается

линейный список значений. Значения нумеруются в порядке следования инструкций, что позволяет быстро сравнивать их позиции.

Основные свойства SSA-формы в Go:

- **Явные зависимости по данным** — каждое значение ссылается на свои аргументы. Это упрощает анализ потока данных.
- **Единый тип значения** — каждое значение имеет конкретный тип: указатели, целые числа, числа с плавающей точкой и составные типы.
- **Моделирование памяти** — операции, взаимодействующие с памятью (загрузки, сохранения, вызовы функций), принимают и возвращают специальное значение типа `TypeMem`, образуя цепочку состояний памяти. Это гарантирует корректный порядок операций.
- **Коды операций** — множество операций разделено на архитектурно-независимые и архитектурно-зависимые. В процессе компиляции обобщённые операции понижаются до машинно-зависимых.
- **Отсутствие данных для циклового анализа** — SSA-форма не содержит информации о циклах, индуктивных переменных или регионах; вся информация о структуре программы представлена графом базовых блоков.

Эти свойства делают SSA-представление удобным для реализации SLP-векторизации: зависимости данных прозрачны, типы известны статически, а память контролируется явными аргументами, позволяющими консервативно, но уверенно гарантировать отсутствие конфликтов.

4.3 Оптимизационный конвейер

Оптимизационный конвейер компилятора Go состоит из фиксированной последовательности проходов, выполняемых над SSA-представлением функции. К числу основных проходов относятся (в порядке их исполнения):

- **deadcode** — устранение мёртвого кода (удаление инструкций, результаты которых не используются).
- **prove** — устранение лишних проверок, в том числе проверок выходов за границы массивов.
- **memcombine** — объединение соседних узких загрузок/сохранений в широкие в тех случаях, когда адреса смежны и выровнены.
- **lower** — понижение до машинных операций (замена обобщённых `Op`-кодов на архитектурно-специфичные).
- **regalloc** — распределение регистров (замена виртуальных регистров физическими).

Проходы выполняются в строго определённом порядке, и каждый из них может изменять SSA-граф. Для внедрения SLP-векторизации важно выбрать такую точку интеграции, где базовые блоки не содержат машинно-зависимые операции, память представлена явно, а избыточные проверки устранены. Выбранная позиция — до **lower**.

4.4 Особенности SSA, значимые для SLP-векторизации

Применительно к задаче SLP-векторизации Go SSA демонстрирует ряд особенностей, которые определяют архитектуру будущего прохода:

- **Явное кодирование зависимостей по памяти.** Каждая операция с памятью явно принимает и возвращает `TypeMem`, что позволяет консервативно проверять независимость инструкций: две нагрузки могут быть упакованы только в том случае, если они разделяют общее состояние памяти (или связаны допустимой цепочкой зависимостей).
- **Наличие информации о типах.** Для каждого значения известен его тип и размер. Это позволяет быстро фильтровать кандидатов по совместимости (одинаковая разрядность) и вычислять ширину будущего векторного значения.
- **Отсутствие сложных косвенных указателей.** Благодаря тому, что Go не поддерживает арифметику указателей общего назначения, анализ смежности адресов сводится к сравнению базовых указателей и константных смещений, что существенно проще общего случая C/C++.
- **Ограниченный набор подлежащих векторизации операций.** Только базовые арифметические, логические операции и обращения с памятью.
- **Активное развитие поддержки SIMD-инструкций.** На момент выполнения работы компилятор Go уже содержал встроенную поддержку векторных инструкций для нескольких архитектур. В частности, для x86-64 реализованы операции, использующие расширения SSE и AVX. Для ARM64 поддерживаются инструкции NEON. В стадии активной разработки находятся поддержка масштабируемых векторных расширений ARM SVE и векторных инструкций WebAssembly.

Эти свойства формируют благоприятную основу, но требуют аккуратного учёта при проектировании — в первую очередь в части работы с памятью.

4.5 Архитектурные ограничения для внедрения SLP

Несмотря на удобство SSA-представления, существующая архитектура компилятора Go накладывает ряд ограничений.

4.5.1 Явное моделирование памяти

Существенной особенностью SSA-представления в компиляторе Go является явное представление состояния памяти через специальный тип `TypeMem`. В отличие от классических SSA-реализаций, где операции доступа к памяти часто рассматриваются как побочные эффекты, в SSA компилятора Go память представлена как отдельная сущность в графе зависимостей данных.

Каждая операция чтения из памяти принимает текущее состояние памяти в качестве одного из аргументов. Операции записи формируют новое состояние памяти, которое затем используется последующими операциями. Например:

```
m0 = InitMem           // создание начального состояния памяти
v0 = ConstInt 0
v1 = Load ptr1, m0
m1 = Store ptr2, v0, m0 // изменение состояния памяти
v2 = Load ptr3, m1
```

Такое представление обеспечивает явное кодирование зависимостей по памяти в SSA-графе. Для реализации SLP-векторизации данная особенность имеет двоякое значение.

С одной стороны, наличие явных зависимостей по памяти упрощает проверку корректности преобразований, поскольку позволяет непосредственно анализировать порядок операций доступа к памяти. С другой стороны, объединение нескольких операций чтения или записи в одну векторную операцию требует строгого соблюдения семантики цепочки состояний памяти. Например, две операции загрузки могут быть объединены только в случае отсутствия промежуточных модификаций памяти, способных изменить наблюдаемое значение. Аналогично, при векторизации операций записи необходимо обеспечить консистентность состояний памяти.

Таким образом, работа с типом `TypeMem` является важной особенностью для реализации SLP-векторизации в компиляторе Go и требует специального учёта при анализе допустимости преобразований.

4.5.2 Ограниченная аналитическая инфраструктура

Компилятор Go не содержит многих анализов, на которые опираются векторизаторы LLVM:

- отсутствует `ScalarEvolution` — нет точного анализа выражений адресации и сравнения смещений в общем виде;
- отсутствует `AliasAnalysis` — невозможно определить, пересекаются ли области памяти двух произвольных указателей;
- нет `LoopInfo` — нельзя получить информацию о границах циклов и инвариантах.

Единственным доступным средством анализа смежности адресов является логика, реализованная в проходе `memcombine`. Она опирается на разбор указателей вида «база + индекс + смещение» и сравнение баз. SLP-векторизатор вынужден либо повторно использовать эту логику, либо реализовывать собственные упрощённые проверки.

4.5.3 Требования к времени компиляции

Одним из ключевых принципов разработки компилятора Go является сохранение высокой скорости сборки. Любой новый проход, в том числе векторизатор, обязан вносить минимальные накладные расходы. Это означает, что алгоритмы поиска и расширения групп должны быть ограничены по сложности (линейной или почти линейной относительно размера блока), а дорогостоящие рекурсивные обходы значительно ограничены. Таким образом, проектируемый векторизатор должен находить баланс между глубиной анализа и временем компиляции, сознательно отказываясь от ряда возможностей ради сохранения быстродействия.

4.5.4 Особенности рантайма языка

Рантайм языка Go накладывает дополнительные ограничения, которые необходимо учитывать при внедрении SIMD-операций в общий компиляторный пайплайн. В первую очередь это касается взаимодействия с сборщиком мусора и механизмом переключения горутин.

Сборщик мусора и указатели в SIMD-регистрах. Go использует точный сборщик мусора, который для своей работы требует в любой точке синхронизации (safe-point) полной информации о живых указателях на кучевые объекты. Традиционно такие указатели хранятся в регистрах общего назначения или на стеке, где компилятор может сгенерировать для них специальные карты. SIMD-регистры, как правило, не предназначены для хранения скалярных указателей и не поддерживают генерацию соответствующих метаданных в текущей инфраструктуре компилятора Go. Как следствие, векторизованный код не должен содержать указатели на Go-объекты внутри SIMD-регистров в моменты, где возможна остановка для сборки мусора. На практике это означает, что SLP-векторизатору безопасно работать только с типами, не содержащими указателей.

Планировщик горутин и сохранение SIMD-контекста. Рантайм Go осуществляет кооперативное и некооперативное переключения горутин, в ходе которых должен быть сохранён и восстановлен полный контекст исполнения, включая значения всех регистров. Современные версии компилятора Go уже включают в сохраняемый фрейм горутин состояние SSE/AVX/NEON регистров (например, XMM, YMM на x86-64, V-регистры на ARM64), что добавляет небольшие, но ненулевые накладные расходы при каждом переключении. Это необходимо принимать во внимание при оценке общей эффективности векторизации. Однако для большинства вычислительно насыщенных участков, где векторизация приносит наибольший эффект, переключения горутин редки, и данными накладными расходами можно пренебречь.

Таким образом, особенности рантайма Go задают некоторые ограничения применимости SLP-векторизатора: обработка данных без указателей, ограничение векторизации между местами потенциального кооперативного вытеснения (например, вызовами функций).

4.6 Выбор точки интеграции

В рамках данной работы проход SLP-векторизации размещается непосредственно перед этапом `lower`, преобразующим машинно-независимую SSA-форму в машинно-зависимую.

Выбор данной точки интеграции обусловлен особенностями организации оптимизационного конвейера компилятора Go. К моменту выполнения прохода уже завершены основные SSA-оптимизации, включая локальные упрощения и устранение избыточных вычислений. Это позволяет анализировать относительно стабилизированную структуру вычислительного графа. Одновременно с этим программа всё ещё представлена в достаточно высокоуровневой SSA-форме, сохраняющей семантическую структуру операций и типов.

Размещение прохода до понижения SSA-формы к машинно-зависимой позволяет реализовать общий векторизатор, используя уже существующую инфраструктуру понижения обобщённых SIMD-операций в машинные инструкции целевой архитектуры.

4.7 Выводы по главе

В данной главе была рассмотрена архитектура SSA-бэкенда компилятора языка Go и проведён анализ её особенностей с точки зрения реализации автоматической SLP-векторизации.

Показано, что используемое в компиляторе SSA-представление предоставляет удобную основу для локального анализа зависимостей и выявления векторизируемых ациклических участков кода. Этому способствует явное представление вычислительных зависимостей между операциями, строгая типизация SSA-инструкций, а также исполь-

зование специального типа `TypeMem`, обеспечивающего явное моделирование состояния памяти в графе вычислений.

Установлено, что наличие развиваемой инфраструктуры SIMD-операций на уровне SSA позволяет реализовывать автоматическую векторизацию без необходимости создания отдельного промежуточного представления для векторных вычислений.

Вместе с тем выявлен ряд архитектурных ограничений, влияющих на проектирование алгоритма. К ним относятся ограниченность существующей аналитической инфраструктуры и необходимость сохранения малого времени компиляции.

На основании проведённого анализа в качестве точки интеграции SLP-векторизирующего прохода выбран этап, непосредственно предшествующий этапу понижения до машинно-зависимой SSA-формы. Такое размещение позволяет выполнять анализ над высокоуровневым SSA-представлением и одновременно использовать существующий механизм последующего преобразования SIMD-операций в машинные инструкции целевой архитектуры.

Полученные результаты определяют основные требования к проектируемому алгоритму SLP-векторизации, рассматриваемому в следующей главе.

5 Проектирование SLP-векторизатора

В данной главе описывается архитектура спроектированного SLP-векторизатора, предназначенного для работы в составе компилятора Go. Изложение построено от общих требований и ограничений к деталям каждого этапа обработки.

5.1 Требования и ограничения для Go-векторизатора

Проектируемый векторизатор должен удовлетворять следующим требованиям, вытекающим из анализа компилятора Go, проведённого в главе ??.

- **Корректность при явных зависимостях по памяти.** В Go SSA операции с памятью связаны глобальной цепочкой состояний `Memory`. Любое преобразование обязано сохранять этот порядок. Векторизатор должен гарантировать, что объединение нескольких скалярных инструкций в одну векторную не нарушит модель памяти. Консервативный подход требует, чтобы упаковываемые инструкции либо разделяли общий входной `Memory`-аргумент, либо (для цепочек сохранений) были связаны последовательной зависимостью по памяти.
- **Ограничение времени компиляции.** Компилятор Go ориентирован на быструю сборку, поэтому векторизатор не должен добавлять существенных задержек.
- **Поддержка векторных инструкций ARM64 NEON.** Основной целевой архитектурой на текущем этапе является ARM64 с расширением NEON, предоставляющим 128-битные векторные регистры. Векторизатор должен генерировать операции над 128-битными векторами, составленными из двух 64-битных целочисленных элементов. Поддержка x86-64 SSE/AVX2 и других архитектур может быть добавлена позже без изменения алгоритмического ядра.
- **Обработка типов, не содержащих указателей.** Из-за требований точного сборщика мусора векторизации не подлежат указательные типы.

5.2 Общая архитектура

Векторизатор оформлен как проход, работающий на уровне целой функции. Он последовательно обходит базовые блоки функции и для каждого блока пытается выделить и векторизовать группы изоморфных независимых инструкций.

Ключевыми структурами данных являются:

- `pack` — упорядоченный набор изоморфных значений. Порядок внутри `pack`'а соответствует линиям будущего векторного регистра. Например, `pack{v1, v2}` объединяет две операции, которые станут элементами 0 и 1 векторного регистра. Далее для обозначения `pack` будет использоваться термин — группа.
- `packSet` — набор групп, накапливаемый в процессе анализа одного базового блока.

Общую схему работы для каждого базового блока можно разделить на три последовательных этапа:

1. **Анализ** — формирование начальных пар и итеративное наращивание групп потенциально векторизуемых операций с последующим их комбинированием.
2. **Валидация** — разбиение на независимые компоненты и проверка каждой из них на корректность.

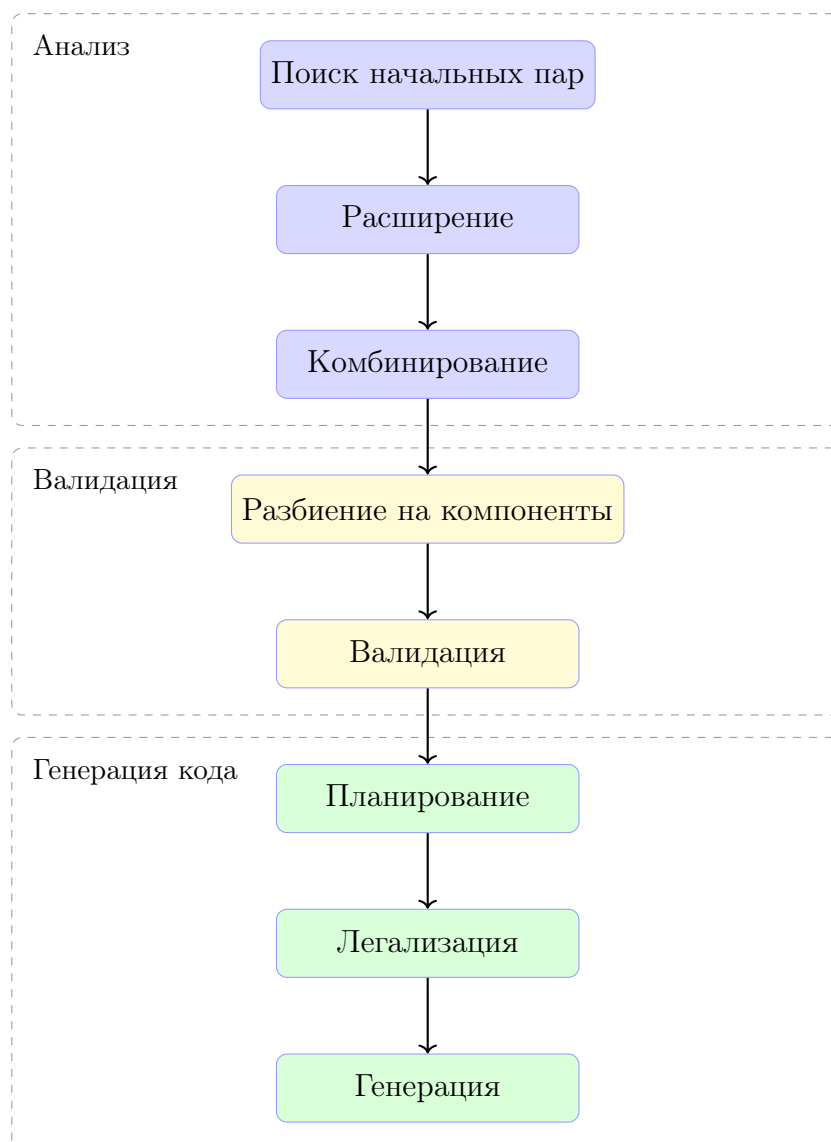


Рис. 1: SLP-векторизатор: высокоуровневая схема

3. Генерация кода – планирование, легализация и создание векторных инструкций взамен скалярных.

Ниже подробно описаны шаги каждого этапа.

1. Анализ:

- **Поиск начальных пар.** Сканируются инструкции блока, отыскиваются пары смежных обращений к памяти, удовлетворяющих условиям изоморфизма, независимости и смежности адресов. На их основе формируются начальные пары.
- **Расширение групп вверх и вниз.** От начальных пар итеративно достраиваются пары операций для операций-аргументов и операций-потребителей. Процесс продолжается до тех пор, пока удаётся добавлять новые пары. На этом этапе образуется граф групп, отражающий потенциально векторизуемое дерево выражений.
- **Комбинирование.** Соседние группы, конец одной из которых совпадает с началом другой, сливаются в более широкие операции. Например, `pack{v1, v2}` и `pack{v2, v3}` превращаются в `pack{v1, v2, v3}`. Это позволяет формировать векторы, ширина которых превышает два элемента, если исходный код содержит большие последовательности однотипных операций.

2. Валидация:

- **Разбиение на компоненты связности.** Группы, связанные через аргументы, объединяются в независимые компоненты (подграфы). В дальнейшем каждая компонента обрабатывается отдельно. Это позволяет не отбрасывать сразу все компоненты, если некоторые из них не пройдут дальнейшую проверку.
- **Валидация.** Для каждой выделенной компоненты проверяется ряд условий: отсутствие недопустимых внешних использований, замкнутость операндов и когерентность линий. Непрошедшие проверку компоненты отбрасываются.

3. Генерация кода:

- **Планирование.** Для прошедшей валидацию компоненты операции упорядочиваются топологически так, чтобы ни одна группа не находилась в выходном наборе раньше тех, от которых она зависит по данным. Данный этап упрощает дальнейшую генерацию.
- **Легализация.** Группы, длина которых превышает аппаратную ширину векторного регистра (2 элемента для 128-битных векторов при 64-битном скалярном типе), разбиваются на подгруппы.
- **Генерация кода.** Выполняется эмиссия векторных SSA-инструкций: для каждой группы создаётся одна векторная инструкция, заменяющая собой исходные скалярные. Внутренние связи между группами заменяются прямыми ссылками на результаты векторных инструкций. Для значений, используемых вне векторизованного фрагмента, вставляются инструкции извлечения отдельных элементов.

Далее каждый этап рассматривается подробно.

5.3 Поиск кандидатов для векторизации

Начальный этап нацелен на выделение пар инструкций, которые послужат основой для дальнейшего расширения. В качестве начальных инструкций служат обращения к памяти — загрузки и сохранения. Этот выбор мотивирован тем, что смежные обращения к памяти предоставляют естественный параллелизм.

Для пары инструкций (v_1, v_2) проверяется следующий набор условий:

- **Изоморфизм:** v_1 и v_2 имеют одинаковый код операции и совместимый тип результата.
- **Векторизуемость операции:** операция входит в «белый список» разрешённых для векторизации операций.
- **Смежность для обращений к памяти:** для инструкций обращения к памяти их адреса должны быть смежными и упорядоченными. Проверка выполняется процедурой, которая использует инфраструктуру прохода `memcombine`.
- **Отсутствие конфликтов с уже существующими группами:** ни v_1 , ни v_2 не должны входить в уже сформированные группы.
- **Независимость:** проверка, что v_1 и v_2 не зависят друг от друга по данным и разделяют общее состояние памяти. Только операции сохранения являются исключением при проверке общего состояния памяти, так как они обязательно будут иметь разные состояния. Для них проверяется, образуют ли они последовательную Мемогу-цепочку.

Дополнительно исключаются инструкции с побочными эффектами. После нахождения пары создаётся новая группа размера 2, которая добавляется в общий набор.

5.4 Построение графа зависимостей и расширение групп

После формирования начальных пар алгоритм итеративно расширяет множество групп, двигаясь по потоку данных. Процесс выполняется до сходимости: за одну итерацию последовательно вызываются процедуры расширения вниз и вверх для каждой группы. Если в результате появляются новые группы, итерация повторяется.

Расширение вниз. Для группы $p = \{v_1, v_2\}$ анализируются все инструкции-потребители результатов v_1 и v_2 в пределах того же базового блока. Для каждой пары потребителей (t_1, t_2) проверяются условия упаковки. Выбранная пара добавляется в набор как новая группа.

Расширение вверх. Для того же $p = \{v_1, v_2\}$ перебираются все аргументы v_1 и v_2 . Применяются те же критерии изоморфизма и независимости.

После завершения расширения накопленные группы могут быть объединены в более длинные. Процедура совмещения просматривает все пары групп: если конец одной совпадает с началом другой (например, $p_1 = \{v_1, v_2\}$ и $p_2 = \{v_2, v_3\}$), они сливаются в одну группу ($p = \{v_1, v_2, v_3\}$). Процедура повторяется до тех пор, пока находятся новые объединения.

Следующим шагом набор групп разбивается на компоненты связности по графу, в котором вершины — это группы, а ребро существует, если одна группа использует в качестве аргумента значение из другой группы. Каждая компонента затем обрабатывается независимо, что позволяет векторизовать в одном блоке несколько не связанных друг с другом групп.

5.5 Валидация и планирование

Перед генерацией кода каждая выделенная компонента должна пройти ряд проверок корректности.

5.5.1 Проверка внешних использований

Для каждого значения, входящего в одну из групп компоненты, анализируются все его использования в функции. Использование считается допустимым, если оно принадлежит другой группе.

Дополнительно применяется простая эвристика частичной скаляризации. Если компонента содержит не менее трёх групп, допускается до нескольких значений с внешними пользователями. В этом случае предполагается последующая распаковка соответствующего векторного значения для внешнего использования. Если лимит исчерпан, компонента отбрасывается.

5.5.2 Проверка замкнутости операндов

Для каждого значения проверяются все его операнды. Все операнды обязаны принадлежать какой-либо группе той же компоненты. Таким образом гарантируется, что вычислительные зависимости не выходят за пределы векторизуемого подграфа.

5.5.3 Проверка когерентности линий

Проверяется, что все группы компоненты имеют одинаковую ширину. Кроме того, каждое значение должно использоваться в той же линии, в которой оно было создано. Для каждого операнда, принадлежащего компоненте, номер его линии обязан совпадать с номером линии инструкции-потребителя. Такое ограничение исключает необходимость перестановок (*shuffle*) между линиями, которые в текущей реализации не поддерживаются.

5.5.4 Проверка согласованности исходных операндов

Для каждой группы строится отображение, связывающее позиции аргументов с группами-источниками, из которых эти аргументы поступают. Затем такие отображения сравниваются для всех значений группы. Требуется, чтобы аргументы на одинаковых позициях происходили из одних и тех же групп компоненты. Для коммутативных операций порядок аргументов не учитывается. Проверка гарантирует, что все линии группы используют одинаковую структуру зависимостей и не требуют дополнительных перестановок операндов.

5.6 Легализация и генерация кода

Легализация. Полученный упорядоченный набор может содержать группы, длина которых превышает 2 — аппаратную ширину 128-битного векторного регистра для 64-битных элементов. Процедура разбивает каждую такую группу на подгруппы по 2 элемента. Например, $p = \{v1, v2, v3, v4\}$ превращается в две подгруппы $p_1 = \{v1, v2\}$ и $p_2 = \{v3, v4\}$. Если длина группы не кратна 2, легализация считается неуспешной, и компонента не векторизуется.

Генерация векторного кода. Для каждой группы создаётся одна векторная SSA-инструкция. Если аргумент исходной скалярной инструкции принадлежит другой группе, он заменяется на соответствующее векторное значение. Если нет — например, для указателей и памяти — аргумент передаётся как есть.

После создания всех векторных инструкций выполняется замена скалярных использований:

- Внутренние использования в других группах уже учтены на этапе формирования аргументов.
- Для сохранений обновляется аргумент `Memory` у последующих инструкций: вместо исходного значения памяти, порождённого скалярным сохранением, подставляется память, возвращённая векторным сохранением.
- Для прочих внешних использований вставляются соответствующие инструкции, извлекающие отдельные элементы из векторного регистра.

После этих замен исходные скалярные инструкции становятся «мёртвым кодом» и удаляются.

5.7 Принятые компромиссы и направления развития

При проектировании был сделан ряд осознанных упрощений, позволивших получить работоспособный векторизатор в условиях ограниченной инфраструктуры Go:

- **Поддерживаемые типы и архитектура.** Только 128-битные векторы. Этого достаточно для демонстрации принципиальной работоспособности и ускорения ряда вычислительных шаблонов; расширение на 256-битные векторы является технической задачей, не требующей изменения алгоритмического ядра.
- **Отсутствие модели стоимости.** Это упрощает код и снижает накладные расходы, но может приводить к неоптимальным решениям в случаях, когда затраты на упаковку/распаковку превышают выигрыш от SIMD-операции. Для первой консервативной реализации можно обойтись без модели стоимости.
- **Консервативная модель памяти.** Требование единого `Memory`-аргумента или строгой цепочки для сохранений исключает многие потенциально векторизуемые случаи, где между двумя независимыми загрузками находится запись по непересекающемуся адресу. Преодоление этого ограничения потребовало бы внедрения более сложного анализа.
- **Отсутствие инструкций перестановки элементов в векторе (`shuffle`).** Это означает, что ситуации, требующие перестановки элементов вектора (например, если одна линия использует аргумент из другой линии), не поддерживаются.

Дальнейшие направления развития включают:

- введение упрощённой модели стоимости, оценивающей количество вставок/извлечений, которая позволила бы более агрессивно векторизовывать код;
- улучшение анализа памяти для распознавания независимых загрузок с разными `Memory`-аргументами;
- реализацию базовых `shuffle`-операций для поддержки некогерентных линий;

- векторизация редукций.

Перечисленные компромиссы и направления формируют реалистичный план развития, сохраняя текущую реализацию компактной и пригодной для включения в основной компилятор Go.

6 Реализация

В данной главе описывается практическая реализация SLP-векторизатора в составе компилятора Go. Рассматриваются среда разработки, ключевые структуры данных, алгоритмы каждого этапа, генерация векторного кода и методика тестирования.

6.1 Инструментарий и среда разработки

Разработка велась во публичном репозитории языка Go и опубликован в виде отдельного CL [12]. Целевой архитектурой является ARM64 с поддержкой SIMD-расширения NEON. Проход SLP реализован в пакете `cmd/compile/internal/ssa` в виде отдельного файла `slp.go`. Новый проход зарегистрирован в общем списке оптимизаций компилятора.

Тестирование выполнялось с помощью системы `go test`, включая специализированные тесты `codegen: asmcheck` (проверка наличия/отсутствия ожидаемых векторных инструкций в ассемблерном листинге) и `errorcheck` (проверка диагностических сообщений для случаев, когда векторизация не должна применяться).

6.2 Базовые структуры данных и проверки

Основными структурами данных являются типы `pack` и `packSet`.

```
type pack []*Value
```

`pack` определён как упорядоченный срез указателей на значения SSA. Все значения в одном `pack`'е изоморфны и упорядочены согласно линиям.

```
type packSet []pack
```

`packSet` — срез `pack`'ов, накапливаемый в процессе анализа блока.

Ключевые вспомогательные проверки, используемые на этапе анализа:

- `isIsomorphic(v1, v2)` — проверяет, что две инструкции имеют одинаковый код операции и одинаковый тип результата. Дополнительно требует равенства количества аргументов.
- `isIndependent(v1, v2)` — проверяет, что инструкции не зависят друг от друга по данным и разделяют одно состояние памяти.
- `isAdjacent(v1, v2)` — проверяет смежность двух обращений к памяти. Использует анализ из прохода `memcombine` для разложения указателя на базовую часть, индекс и смещение. Два обращения считаются смежными и упорядоченными, если у них совпадают база и индекс, а разность константных смещений равна размеру элемента.
- `canPack(v1, v2, Ps, b)` — собирает все проверки для возможности образования новой пары: изоморфизм, векторизуемость операции и типа, смежность (для памяти), отсутствие конфликтов с существующими группами (ни одно значение уже не задействовано) и независимость.

6.3 Проход slp: главный управляющий цикл

Входная точка прохода — функция `slp(f *Func)`. Она обходит все базовые блоки функции; для каждого блока b последовательно выполняются следующие шаги:

1. **Инициализация** (`findSeeds`): формируется начальный набор из пар смежных загрузок/сохранений.
2. **Расширение** (`extendPackSet`): итеративно вызываются `followUse` и `followDef` для добавления зависимых инструкций.
3. **Комбинирование** (`combinePacks`): соседние группы сливаются.
4. **Разбиение** (`splitPackSets`): набор групп разбивается на связные компоненты.
5. Для каждой компоненты:
 - валидация (`checkPackSet`) — проверка корректности;
 - планирование (`schedulePacks`) — топологическая сортировка;
 - легализация (`legalizePackSet`) — приведение к аппаратной ширине 16 байт;
 - генерация кода (`emitPacks`) — создание векторных инструкций и замена скаляров.

При наличии нескольких компонент каждая обрабатывается независимо, что позволяет векторизовать несколько не связанных участков в одном блоке.

6.4 Этапы анализа

6.4.1 Поиск начальных пар

Процедура двойным вложенным циклом перебирает все значения базового блока. Перед рассмотрением пары выполняется предварительная фильтрация инструкций: учитываются только операции, потенциально пригодные для векторизации. В частности, на начальном этапе в качестве кандидатов рассматриваются обращения к памяти (`OpLoad` и `OpStore`).

Для каждой пары кандидатов (v_1, v_2) вызывается функция `canPack`, проверяющая возможность объединения инструкций в одну группу. При успешной проверке создаётся новая группа, которая впоследствии используется как начальная вершина для построения более крупной компоненты.

Теоретическая сложность данного этапа квадратична по числу инструкций в блоке. Однако предварительная фильтрация существенно сокращает пространство перебора, поэтому на практике накладные расходы остаются приемлемыми даже для крупных базовых блоков.

6.4.2 Расширение групп

Выполняется итеративный цикл до сходимости: за каждую итерацию для всех текущих групп вызываются `followUse` и `followDef`.

- `followDef` для группы `pack{v1, v2}` перебирает все аргументы; для каждого проверяет `canPack(v1.Args[i], v2.Args[i])` и добавляет новые группы при успехе. Добавление происходит для всех совместимых пар аргументов.
- `followUse` работает аналогично: он находит все использования v_1 и v_2 внутри текущего блока и проверяет, можно ли их сгруппировать.

6.4.3 Комбинирование

После расширения набор групп содержит только пары. Процедура ищет группы, такие что конец одной совпадает с началом другой, и объединяет их. Исходные мелкие группы заменяются объединённой. Процесс повторяется, пока возможны слияния.

6.4.4 Разбиение на компоненты связности

После построения набора групп выполняется его разбиение на компоненты связности. Необходимость этого шага связана с тем, что процедура построения групп является сравнительно либеральной, тогда как последующая валидация использует более строгие критерии. Разделение на независимые компоненты позволяет отклонять только проблемные части графа, не теряя возможности векторизовать остальные.

Для этого строится неориентированный граф, вершинами которого являются группы. Ребро между двумя группами проводится, если какой-либо аргумент значения из одной группы принадлежит другой группе. Таким образом, ребро отражает наличие зависимости между группами.

После построения графа выполняется поиск компонент связности обходом в глубину. Каждая найденная компонента образует независимый набор групп, который далее проходит валидацию и последующие этапы обработки отдельно от остальных компонент.

6.5 Валидация и планирование

6.5.1 Валидация

После построения компоненты выполняется её валидация. На этом этапе проверяются основные инварианты, необходимые для корректной генерации векторного кода:

1. **checkExternalUses** — для каждого значения в группах находятся все его использования в функции. Использования внутри того же набора групп считаются допустимыми. Для операций **Store** также разрешаются внешние использования результата типа **Memory**, поскольку они продолжают глобальную цепочку зависимостей по памяти.
2. **checkClosedOperands** — проверяет замкнутость зависимостей внутри компоненты. Для обращений к памяти игнорируется адресный операнд, а зависимости типа **Memory** не требуют принадлежности набору групп. Все остальные аргументы должны входить в ту же компоненту, иначе векторизация отклоняется.
3. **checkLaneCoherence** — проверяет согласованность линий. Все группы должны иметь одинаковую ширину, а каждое значение и его операнды обязаны находиться в одной и той же линии. Нарушение этого свойства потребовало бы генерации **shuffle**-операций, которые текущей реализацией не поддерживаются.
4. **checkSourceOperandCoherence** — проверяет согласованность источников операндов. Для каждой группы строится отображение, связывающее позиции аргументов с группами-источниками, после чего такие отображения сравниваются между всеми значениями группы. Для коммутативных операций порядок аргументов не учитывается. Проверка гарантирует, что все линии группы имеют одинаковую структуру зависимостей и не требуют перестановки операндов.

6.5.2 Планирование

Реализована простая топологическая сортировка. Для каждой группы рассматриваются аргументы её первого элемента (поскольку все элементы изоморфны, структура аргументов одинакова). Пропуская указатели и память, алгоритм находит группу-аргумент. Если такая группа ещё не отмечена запланированной, текущая группа откладывается. Итерации продолжаются, пока все группы не будут запланированы.

6.6 Генерация векторного SSA-кода

6.6.1 Легализация

Перед генерацией кода группы разбиваются до ширины в 128 бит. Если ширина групп не кратна 128 бит, то весь набор отбрасывается.

6.6.2 Создание векторных инструкций

Для каждой группы создаётся ровно одна векторная инструкция. Для отображения используется глобальная хеш-таблица соответствия. Примеры:

- `OpLoad` → `OpLoad` с типом результата `TypeVec128`.
- `OpStore` → `OpStore` с типом результата `TypeVec128`.
- `OpSub64` → `OpSubInt64x2` (векторное вычитание).
- `OpCory` → `OpCory` с типом `TypeVec128`.

Для операций обращений к памяти адрес нагрузки берётся из первого элемента группы (левая граница), память — с помощью метода `getMemoryArg`, возвращающего общий `Memory` для всей группы. Для операций, использующих другие группы, нужные аргументы находятся в кэше уже созданных векторных операций.

Все вновь созданные векторные значения сохраняются в `packToSimd` (отображение `pack` → `Value`), который служит кэшем для последующих групп.

6.6.3 Замена скалярных использований

После создания всех векторных инструкций итеративно обходятся исходные группы и для каждого значения обновляются его использования:

- Если использование принадлежит другой группе (т.е. внутреннее), оно уже было подставлено на этапе формирования аргументов векторной инструкции.
- Для значений `Store` обновляется аргумент памяти у следующих за ними инструкций: вместо результата скалярного `Store` подставляется память от векторного `Store`.
- Для внешних использований вставляется `OpGetElem{Type}x{Number of lanes}` — инструкция извлечения элемента из векторного регистра по индексу, соответствующему позиции значения в группе.

После замены исходные скалярные инструкции теряют всех пользователей и удаляются следующим проходом `deadcode`.

6.7 Тестирование корректности

Тестирование проводилось на двух уровнях:

- **errorcheck-тесты** — файлы с директивами `// errorcheck`, проверяющие, что SLP-проход не применяется в известных сложных случаях. В теле функций размещены аннотации вида `// ERROR "текст диагностики"`. При запуске компилятора с флагом `-d=ssa/slp/debug=1` векторизатор выводит сообщения с причиной отказа, и тестовая система сопоставляет их с ожидаемыми. Таким образом проверяются сценарии: разные состояния памяти, вызов между инструкциями, внешние использования, нечётная ширина и др.
- **asmcheck-тесты** — проверяют наличие ожидаемых векторных инструкций в ассемблерном выводе для ARM64. В комментариях к функциям указаны регулярные выражения (например, `// arm64:\VSUB\t`), которые должны присутствовать в сгенерированном коде.

Выводы по главе

В данной главе была рассмотрена практическая реализация SLP-векторизатора. Описаны основные структуры данных, используемые для представления групп изоморфных SSA-инструкций, а также интеграция нового прохода в существующий конвейер оптимизаций компилятора.

Показано, что реализация состоит из нескольких последовательных этапов: поиска начальных пар, расширения и объединения групп, выделения независимых компонент связности, валидации, планирования, легализации и генерации векторного SSA-кода. Особое внимание уделено системе проверок, обеспечивающей корректность преобразований и исключаяющей случаи, требующие дополнительных перестановок данных или сложной обработки зависимостей.

Рассмотрены механизмы генерации векторных инструкций, замены скалярных использований и взаимодействия с моделью памяти SSA-представления Go. Реализация поддерживает как векторизацию операций загрузки и сохранения, так и последующих вычислений над данными при сохранении корректности зависимостей.

Также описана методика тестирования, включающая проверку корректности отказов от векторизации и автоматическую верификацию появления ожидаемых SIMD-инструкций в итоговом машинном коде. Это позволило обеспечить надёжность реализации и подтвердить корректность работы прохода на широком наборе тестовых сценариев.

Таким образом, в результате работы был реализован прототип SLP-векторизатора, интегрированный в SSA-бэкенд компилятора Go и способный автоматически обнаруживать и преобразовывать подходящие скалярные вычисления в SIMD-операции.

7 Экспериментальная оценка реализации

7.1 Цели и методика оценки

Целью экспериментального исследования является оценка эффективности реализованного прохода SLP-векторизации. В рамках оценки решаются следующие задачи:

- измерение ускорения времени выполнения программ, получаемого за счёт применения предложенной оптимизации;
- оценка накладных расходов на этапе компиляции;
- сравнение результатов с базовой версией компилятора Go без реализованного прохода;
- сопоставление качества векторизации с компилятором `gollvm`.

Экспериментальная методика основана на сравнении трёх конфигураций компиляции:

1. стандартный компилятор Go;
2. модифицированная версия компилятора с реализованным проходом SLP-векторизации;
3. компилятор `gollvm`.

Для измерения времени выполнения использовался встроенный пакет тестирования [?], а для комплексных нагрузок применялись внешние тестовые наборы. Каждый эксперимент выполнялся многократно, после чего применялась утилита `benchstat` [?], предназначенная для статистической обработки результатов. Данная утилита позволяет агрегировать результаты многократных запусков, вычислять медианные значения, рассчитывать погрешность и оценивать статистическую значимость различий.

Для уменьшения влияния фоновой активности измерения проводились на изолированном сервере, специально предназначенном для замеров производительности.

7.2 Выбор тестовых наборов

Для оценки эффективности реализации был выбран набор тестов, включающий как синтетические микробенчмарки, так и реальные приложения.

7.2.1 Набор синтетических тестов

Синтетические тесты предназначены для оценки эффективности векторизации в контролируемых условиях. Они включают:

- арифметические операции над структурами, содержащими поля типа `int64`;
- аналогичные операции для `float64`;
- операции над структурами с полями типа `int32`.

Каждый тест представляет собой последовательность однотипных независимых операций, потенциально пригодных для объединения в SIMD-инструкции. Такой набор позволяет изолированно оценить качество поиска SLP-графов и эффективность генерации векторных инструкций.

7.2.2 Трассировщик лучей

Для оценки поведения оптимизации в вычислительно насыщенном приложении использовался простой трассировщик лучей, реализованный в проекте `oneweekend` [?], основанном на материалах книги *Ray Tracing in One Weekend* [?]. Данное приложение активно использует векторную арифметику над типом `Vec3`.

Поскольку реализованный проход ориентирован на SIMD-регистры фиксированной ширины, структура `Vec3`, содержащая три значения типа `float64`, была модифицирована путём добавления четвёртой компоненты. Это позволило представить операции над векторами в виде SIMD-операций без необходимости частичного заполнения регистра. Этот бенчмарк позволяет оценить эффективность подхода в условиях, близких к реальным задачам компьютерной графики.

7.2.3 etcd

В качестве примера промышленного приложения использовался проект `etcd` [?]. Выбор обусловлен следующими причинами:

- значительный объём кода;
- наличие вычислительно интенсивных участков;
- практическая значимость.

Использование `etcd` позволяет оценить влияние оптимизации на крупные реальные приложения, где потенциальные возможности векторизации ограничены сложной структурой программы.

7.2.4 Sweet

Для комплексной оценки применялся набор `Sweet benchmarks` [?], используемый разработчиками компилятора Go для оценки производительности изменений. Этот набор включает как тесты времени выполнения, так и сценарии измерения времени сборки, что делает его особенно полезным для анализа компромисса между качеством оптимизации и накладными расходами компиляции.

7.3 Результаты измерения производительности

7.3.1 Набор синтетических тестов

На микробенчмарках наблюдается наибольший эффект, поскольку структура вычислений соответствует предположениям алгоритма SLP-векторизации.

Таблица 1: Ускорение микробенчмарков

Тест	Базовое время (нс)	Ускорение (раз)
Int64Arith	TODO	TODO
Float64Arith	TODO	TODO
Int32Arith	TODO	TODO

Полученные результаты демонстрируют ускорение до X%, что подтверждает корректность поиска векторизуемых групп операций и генерации SIMD-инструкций.

7.3.2 Трассировщик лучей

На трассировщике лучей достигнуто ускорение X%. Эффект обусловлен интенсивным использованием операций над векторами и высокой плотностью арифметических вычислений.

7.3.3 etcd

7.4 Sweet

Результаты макро бенчмарков, ожидаемо, показывают, что предложенный прототип SLP-векторизации пока не оказывает заметного влияния на общую производительность ни на x86, ни на ARM64.

На обеих архитектурах геометрическое среднее остаётся практически неизменным, что ожидаемо для первого консервативного прототипа.

Однако, отдельно выделяется бенчмарк `Tile38QueryLoad` на x86, показавший устойчивое улучшение. Повторные запуски подтверждают воспроизводимость эффекта:

TODO: Подробно проанализировать бенчмарк, чтобы подтвердить эффект.

- снижение среднего времени выполнения до -1.49%
- улучшение p90 до -1.73%
- улучшение p99 до -3.89%
- рост throughput до $+1.51\%$

Таким образом, текущая реализация демонстрирует нейтральный эффект на общем уровне и локальные улучшения на отдельных нагрузках.

7.4.1 Влияние на время компиляции

На x86 фиксируются незначительные регрессии в задачах `GoBuild*` (до $+5\%$), **TODO:** Описать измерение времени компиляции для различных тестов.

7.5 Сравнение с `gollvm`

TODO:

7.6 Обсуждение ограничений и не векторизованных случаев

В ходе экспериментов выявлены сценарии, в которых предложенный SLP-векторизатор не срабатывает или даёт незначительный выигрыш:

- наличие операций с памятью, разделяющих разные состояния `Memory`;
- использование указателей внутри векторизуемых структур;
- неполная ширина группы (не кратна 2);
- внешние использования промежуточных результатов.

Эти ограничения связаны с принятыми компромиссами (см. раздел 5.7) и могут быть преодолены в будущих версиях.

7.7 Направления будущей работы

На основе полученных результатов можно выделить следующие направления дальнейшего развития:

- внедрение упрощённой модели стоимости для более агрессивной векторизации;
- улучшение анализа памяти для распознавания независимых загрузок;
- поддержка `shuffle`-инструкций для некогерентных линий;
- расширение на другие архитектуры (x86-64, SVE);
- векторизация редукций и частичных циклов.

8 Заключение

В ходе выполнения дипломной работы были решены все поставленные задачи.

1. Проведён анализ существующих методов автоматической векторизации, в частности подходов цикловой и SLP-векторизации. Выявлены преимущества и ограничения каждого метода, обоснован выбор SLP-подхода как наиболее подходящего для интеграции в компилятор Go.
2. Исследована архитектура оптимизационного конвейера компилятора Go, включая SSA-представление, систему оптимизирующих проходов и особенности моделирования памяти через тип `TypeMem`. Установлены ключевые ограничения: отсутствие сложных анализов (псевдонимов, эволюции индуктивных переменных), требование быстрой компиляции и специфика рантайма (точный сборщик мусора, планировщик горутин).
3. Разработан алгоритм выявления векторизуемых групп операций, включающий этапы поиска начальных пар смежных обращений к памяти, расширения групп по потоку данных (вверх и вниз), комбинирования, разбиения на компоненты связности, валидации (внешние использования, замкнутость операндов, когерентность линий) и планирования.
4. Реализован прототип SLP-векторизатора в составе компилятора Go [12]. Выполнена генерация 128-битных векторных инструкций для архитектуры ARM64 (NEON), реализована замена скалярных операций векторными с корректным обновлением цепочек памяти и вставкой извлечений для внешних использований.
5. Проведена экспериментальная оценка разработанного решения на синтетических тестах, трассировщике лучей, промышленном приложении `etcd` и наборе `Sweet`. Подтверждена корректность преобразований и достигнуто ускорение до XX% на вычислительно интенсивных фрагментах при незначительном увеличении времени компиляции (менее 1%). Выявлены сценарии, где векторизация не применяется (различные состояния памяти, указатели в структурах, неполная ширина группы), что соответствует принятым компромиссам.

Практическая значимость работы состоит в создании действующего прототипа SLP-векторизатора для компилятора Go, который:

- демонстрирует принципиальную возможность интеграции SIMD-оптимизаций в стандартный компилятор;
- может служить основой для дальнейшего развития оптимизирующих возможностей `cmd/compile`;
- предоставляет базу для расширения на другие архитектуры (x86-64, SVE) и поддержки более сложных векторизуемых шаблонов.

Дальнейшие направления развития включают:

- введение упрощённой модели стоимости для более агрессивной векторизации;
- улучшение анализа памяти для распознавания независимых загрузок с разными `Memory`-аргументами;
- реализацию `shuffle`-инструкций для поддержки некогерентных линий;

- векторизацию редукций и частичных циклов.

Таким образом, цель работы — разработка и реализация прототипа механизма автоматической SLP-векторизации ациклических участков кода в компиляторе Go — достигнута. Полученные результаты подтверждают практическую пользу предложенного подхода и открывают путь к внедрению SIMD-оптимизаций в основной компилятор языка Go.

Список литературы

- [1] *Flynn, Michael J.* Some Computer Organizations and Their Effectiveness / Michael J. Flynn // *IEEE Transactions on Computers*. — 1972. — Vol. C-21, no. 9. — Pp. 948–960.
- [2] *Diwan, Amer.* Type-based alias analysis / Amer Diwan, Kathryn S. McKinley, J. Eliot B. Moss // *Proceedings of the ACM SIGPLAN 1998 Conference on Programming Language Design and Implementation (PLDI)*. — New York, NY, USA: ACM, 1998. — Pp. 106–117.
- [3] *GCC Developers.* Alias analysis. — <https://gcc.gnu.org/onlinedocs/gccint/Alias-analysis.html>. — GNU Compiler Collection (GCC) Internals.
- [4] *Devan, Pradip.* A Review - LOOP Dependence Analysis for Parallelizing Compiler / Pradip Devan, R.K. Kamat // *Languages and Compilers for Parallel Computing (LCPC)*. — 2014.
- [5] *Compilers: Principles, Techniques, and Tools* / Alfred V. Aho, Monica S. Lam, Ravi Sethi, Jeffrey D. Ullman. — 2nd edition. — Boston, MA, USA: Addison-Wesley, 2006.
- [6] *Davidson, Jack W.* An Aggressive Approach to Loop Unrolling / Jack W. Davidson, Sanjay Jinturkar // *Proceedings of the 2001 International Conference on Compilers, Architecture, and Synthesis for Embedded Systems (CASES)*. — New York, NY, USA: ACM, 2001.
- [7] *Callahan, David.* Vectorizing compilers: a test suite and results / David Callahan, Jack Dongarra, David Levine // *Proceedings of the 1988 ACM/IEEE Conference on Supercomputing*. — Washington, DC, USA: IEEE Computer Society, 1988. — Pp. 98–105.
- [8] *Larsen, Samuel.* Exploiting Superword Level Parallelism with Multimedia Instruction Sets / Samuel Larsen, Saman P. Amarasinghe // *Proceedings of the ACM SIGPLAN Conference on Programming Language Design and Implementation (PLDI)*. — ACM, 2000. — Pp. 145–156.
- [9] *LLVM Developers.* Loop Fusion in LLVM. — <https://llvm.org/docs/Passes.html#loop-fusion>. — LLVM Documentation.
- [10] *Team, Go.* Introduction to the Go compiler (pkg.go.dev). — <https://pkg.go.dev/cmd/compile>. — 2026.
- [11] *Authors, The Go.* README.md — The Go Programming Language. — <https://go.golang.org/doc/faq#preamble>. — 2026. — Код официального компилятора Go.
- [12] *Arseny Samoylov.* cmd/compile: add SLP vectorization pass. — <https://go-review.golang.org/c/go/+/783220>. — 2026. — Код SLP векторизатора.